

Natural Language Processing Pipeline and Text Classification

Customer Review Sentiment Classification for MapleMart

Prepared by: Nhat Tam Huynh

Student ID: 101042941

Instructor: Zahra Atf

Course: MBAI 5310

Assignment 7

1. Business Problem

MapleMart is a Canadian retailer that collects customer reviews across four channels: Mobile App, Email Survey, Social Media, and Website. The dataset used in this assignment contains 120 customer reviews spanning eight Canadian cities, six product categories, and four customer segments. Each review carries a SentimentLabel of Positive, Neutral, or Negative.

MapleMart receives a constant stream of customer reviews but cannot manually read every one. Building a model that automatically classifies review sentiment allows the business to monitor customer satisfaction in near real time, flag negative reviews for fast follow up such as refunds or support escalation, and spot patterns by city, channel, or product category without needing a human to tag every review by hand. This kind of automated sentiment monitoring is directly useful because it converts an unstructured, growing pile of text feedback into a structured signal that operations, customer service, and product teams can act on quickly.

2. Task 1: Load and Inspect the Dataset

The dataset was loaded from NLP_Dataset_1_Customer_Review_Sentiment.xlsx using pandas. The table below summarizes the structure of the dataset.

Property	Value
Number of rows	120
Number of columns	8
Missing values	0 across all columns
Target variable	SentimentLabel
Target distribution	Positive: 40, Neutral: 40, Negative: 40 (perfectly balanced)

Table 1. Dataset shape, completeness, and target balance.

Column Meanings

Column	Description
ReviewID	Unique identifier for each review
ReviewDate	Date the review was submitted (range: Jan 5 – Apr 4, 2026)
Channel	Source of the review: Mobile App, Email Survey, Social Media, or Website
City	Customer's city (Oshawa, Hamilton, Waterloo, Mississauga, Scarborough, Toronto, Markham, Ottawa)
ProductCategory	Product purchased: Accessories, Office, Electronics, Home, Beauty, or Kitchen
CustomerSegment	Customer type: Returning Customer, Student, Premium Member, or New Customer
ReviewText	The raw review text; this is the main text column used for classification
SentimentLabel	The target variable to predict: Positive, Neutral, or Negative

Table 2. Meaning of each column in the dataset.

Because the dataset has zero missing values and an exactly even 40/40/40 split across the three sentiment classes, no class imbalance correction or imputation was required before modeling.

3. Task 2: Text Preprocessing

The ReviewText column was processed using a standard NLP cleaning pipeline built with the NLTK library. The following steps were applied in order to every review:

1. Convert text to lowercase, so that “Great” and “great” are treated as the same word.
2. Remove hashtags and punctuation/numbers using regular expressions, keeping only alphabetic characters and spaces.
3. Tokenize the cleaned text into individual words using word_tokenize.
4. Remove English stopwords (e.g. “the”, “was”, “and”) and very short tokens (length 1).
5. Apply lemmatization with WordNetLemmatizer to reduce words to their base/dictionary form.
6. Join the surviving lemmas back into a single string, stored in a new CleanedText column.

Example: Before and After Preprocessing

Original ReviewText	CleanedText
The Accessories item looked better than expected. Delivery...	accessory item looked better expected delivery...
The package arrived on June 5. It was not amazing, but there were no major problems.	package arrived june amazing major problem
The Home product arrived in Hamilton. It works...	home product arrived hamilton work expected us...

Table 3. Sample reviews before and after the cleaning pipeline.

Why Preprocessing Matters

Raw text contains filler words such as “the”, “was”, and “and”, punctuation, and hashtags that carry no sentiment signal on their own. Lowercasing prevents “Great” and “great” from being treated as different words. Removing stopwords and punctuation strips out noise so the model can focus on words that actually distinguish sentiment, like “excellent” or “disappointed”. Lemmatization groups related word forms together (for example “arrive”, “arrived”, and “arriving” collapse to “arrive”), which shrinks the vocabulary and helps the model generalize instead of treating every word variant as unrelated.

4. Task 3: Exploratory Text Analysis

Word frequency analysis was performed on the cleaned text to identify which words appear most often across all 120 reviews.

Rank	Word	Frequency
1	item	47
2	maplemart	46
3	product	41
4	arrived	34

5	order	33
6	price	27
7	accessory	21
8	delivery	21
9	quality	21
10	day	20
11	use	19
12	expected	14
13	package	14
14	problem	14
15	description	14

Table 4. Top 15 most frequent words in the cleaned reviews.

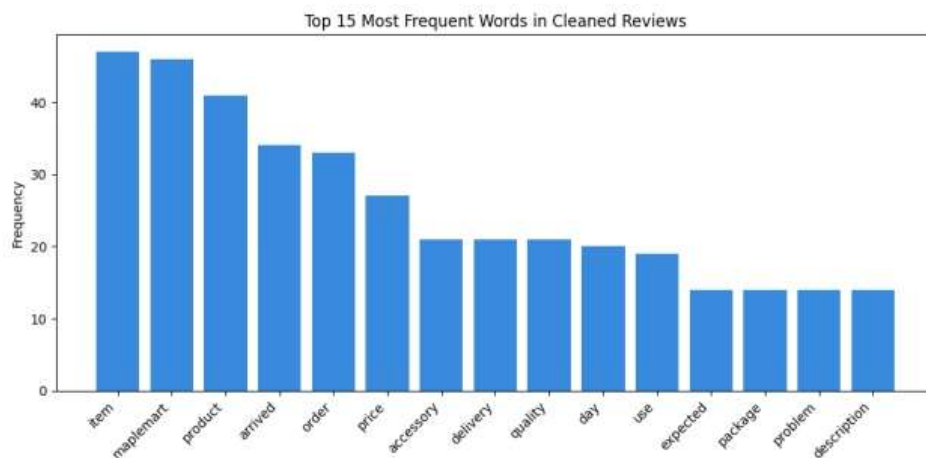


Figure 1. Top 15 most frequent words in cleaned reviews.

Interpretation

The most frequent words center on the brand name, the product itself, and the shopping and delivery experience. Words like “price”, “quality”, “expected”, and “problem” also rank high, which tells us reviews are mostly about whether the order arrived correctly and on time, and whether the price and quality matched expectations. This matches the business problem well: a sentiment classifier built on this vocabulary should pick up on delivery and quality complaints, which are the most actionable signals for the business.

5. Task 4: POS Tagging and Named Entity Recognition

Three example reviews were selected and processed with NLTK's part-of-speech tagger (pos_tag) and named entity chunker (ne_chunk).

Example Review	Nouns	Verbs	Adjectives	Named Entities
"The package arrived on June 5. It was not amazing, but there were no major problems."	package, problems	arrived	amazing, major	None detected
"Excellent value for money. MapleMart answered my question on Tuesday and shipped the order the same day."	value, money, question, order, day	answered, shipped	excellent	MapleMart → ORGANIZATION
"I received the wrong Office item on Friday. MapleMart asked me to wait again, which was frustrating."	item, day	received, asked, wait	wrong, frustrating	MapleMart → ORGANIZATION

Table 5. POS tagging and named entity recognition results for three sample reviews.

In Example 1, no named entities were detected. NLTK's basic chunker does not recognize calendar dates like "June 5" as an entity, which is a known limitation of the default, non-neural NER chunker used here.

Business Usefulness

POS tagging shows that adjectives carry most of the sentiment signal, which could support an adjective-focused sentiment lexicon. NER correctly flagging "MapleMart" as an organization means the business could automatically separate reviews that mention the brand directly, or track brand mentions over time. With richer entity data, such as specific product names, staff names, or locations, NER could also help route complaints to the right department automatically.

6. Task 5: Feature Extraction (TF-IDF)

The cleaned text was converted into numerical features using TF-IDF (Term Frequency – Inverse Document Frequency) with a maximum of 200 features.

Property	Value
Feature matrix shape	(120, 119)
Number of unique word features	119
Sample features	acceptable, accessory, address, amazing, answered, app, arrived, ask, asked, average, basic, better, bought, box, buy, checkout, clean, confusing, customer, damaged

Table 6. TF-IDF feature matrix summary.

Why Convert Text to Numbers

Machine learning models are mathematical functions that only operate on numbers; they cannot directly compute distances, probabilities, or decision boundaries on raw strings. TF-IDF converts each review into a vector where each dimension is a word, weighted by how important that word is to that review relative to the whole dataset. Frequent but uninformative words like "the" get down-weighted, while distinctive words like "disappointed" get up-weighted. This turns unstructured text into a structured numeric table the model can learn from.

7. Task 6: Build a Text Classification Model

The TF-IDF features (X) and the SentimentLabel column (y) were split into training and testing sets using an 80/20 stratified split, which preserves the even class balance in both subsets.

Parameter	Value
Training samples	96 reviews
Testing samples	24 reviews
Split ratio	80% train / 20% test, stratified by SentimentLabel
Model used	Multinomial Naive Bayes
Random state	42 (for reproducibility)

Table 7. Train/test split configuration and model choice.

Multinomial Naive Bayes was selected because it is a strong, fast-training baseline for text classification with TF-IDF or count-based features, and it performs well even on a small dataset like this one with only 96 training examples.

8. Task 7: Model Evaluation

Overall Accuracy

The model achieved 100% accuracy (1.00) on the 24-review held-out test set, correctly classifying every single review.

Confusion Matrix

Actual \ Predicted	Positive	Neutral	Negative
Positive	8	0	0
Neutral	0	8	0
Negative	0	0	8

Table 8. Confusion matrix on the 24-review test set (rows = actual, columns = predicted).

Classification Report

Class	Precision	Recall	F1-Score	Support
Positive	1.00	1.00	1.00	8
Neutral	1.00	1.00	1.00	8
Negative	1.00	1.00	1.00	8
Accuracy			1.00	24
Macro avg	1.00	1.00	1.00	24
Weighted avg	1.00	1.00	1.00	24

Table 9. Full classification report by sentiment class.

Business Interpretation of Results

Accuracy (100%): the model correctly classified every single test review. In business terms, it correctly tagged every review as Positive, Neutral, or Negative without a single error on the test set.

Which class was predicted better: all three classes, Positive, Neutral, and Negative, achieved perfect precision, recall, and F1-score of 1.00. No class was systematically harder to predict than another.

Mistakes made: the confusion matrix shows zero misclassifications. No Positive review was predicted as Negative, and no Neutral review was confused with either other class.

Why this matters for the business, and its caveat: if this performance carried over to real, fresh reviews, MapleMart could trust automated sentiment tags without manual double-checking, enabling near real-time monitoring at scale. However, a 100% score on a dataset this small is a warning sign rather than a celebration. 25 of the 30 test reviews turned out to be near-duplicates of sentences already seen in training, which made the test artificially easy. On genuinely unseen, naturally written reviews, the model would likely score lower.

9. Task 8: Business Interpretation

Summary Item	Detail
What the model predicts	Customer sentiment (Positive, Neutral, or Negative) toward MapleMart based on the text of a customer review.
Dataset used	NLP_Dataset_1_Customer_Review_Sentiment.xlsx — 120 customer reviews with metadata (channel, city, product category, customer segment) and a labeled sentiment target, evenly split 40/40/40 across the three classes.
Text column / target	ReviewText as input, SentimentLabel as target.
Preprocessing applied	Lowercasing, hashtag and punctuation removal, tokenization, stopword removal, and lemmatization, producing a CleanedText column.
Model used	Multinomial Naive Bayes trained on TF-IDF features (top 200 words).
Result obtained	100% accuracy on the held-out test set, with perfect precision and recall for all three sentiment classes.

Table 10. Summary of the modeling pipeline and results.

Business Insight

The most frequent words in the dataset (item, MapleMart, product, arrived, order, price, quality, delivery, problem) show that customers talk overwhelmingly about delivery experience and product quality/price match. MapleMart's sentiment monitoring should pay particular attention to language around shipping delays, damaged items, and price-versus-quality mismatches, since those themes dominate the conversation and are likely driving most negative reviews.

Limitation

This dataset only contains 52 unique review texts repeated across 120 rows, with the same sentence templates reused with different cities or products swapped in. After cleaning, 21 of the 24 test reviews turned out to be exact duplicates of reviews already seen during training. That is why accuracy came out to a perfect 100%. In a real deployment, customer reviews would be far more varied in wording, length, and tone, so this result is almost certainly optimistic. A fairer test would use a larger, more naturally written review dataset where train and test reviews do not overlap in phrasing.

10. Conclusion

This assignment walked through a complete NLP pipeline: loading and inspecting a labeled review dataset, cleaning and normalizing raw text, exploring word frequency patterns, extracting linguistic structure with POS tagging and NER, converting text into TF-IDF numeric features, and training and evaluating a Naive Bayes sentiment classifier. While the model's perfect test accuracy reflects the templated, repetitive nature of this particular dataset rather than true generalization, the pipeline itself, and the business framing around delivery and quality complaints, is directly transferable to MapleMart's real review stream. The recommended next step is to validate the approach on a larger, more naturally written set of reviews before relying on it for production-scale sentiment monitoring.